CS 422 — Machine Learning

Midterm Reviewer

Concepts, Principles & Practical Reasoning — 2nd Semester AY 2025–2026

Topic 1 — Data Splitting & Class Imbalance

The Core Problem

When a dataset has far more examples of one class than another — for instance a medical dataset where only a small fraction of patients are high-risk — a naive random split can accidentally concentrate most of the minority class in just one subset. The other subsets end up almost completely lacking that class, which distorts both training and evaluation.

Stratified Splitting

Stratification is the practice of preserving the original class ratio in every subset produced by a split. Think of it like distributing a deck of cards: instead of cutting randomly, you deal proportionally so each pile has the right mix of suits.

Aspect	Detail
What it does	Ensures each split mirrors the class distribution of the full dataset.
When to use it	Whenever one class is rare or the dataset is small enough that random chance could skew proportions.
Key parameter	stratify=y in scikit-learn's train_test_split — must be applied independently to every split in a multi-stage split (e.g., train/val/test).
Reproducibility	random_state pins the random seed so the same split is produced every run. Always set it for reproducible experiments.

Multi-Stage Splits

Splitting into three sets (train, validation, test) requires two passes. Common proportions: 60% train, 20% val, 20% test. The usual recipe: first split off 40%, then cut that 40% in half. Crucially, stratification must be re-applied at each pass — it does not carry over automatically.

💡 If you stratify the first split but forget the second, the validation or test set may end up with skewed class proportions, silently undermining your evaluation.

📌 Splitting X and y together in one call is mandatory. Splitting them in separate calls de-synchronises rows and produces incorrect label-feature pairings.

Topic 2 — Validation Set vs. Test Set

Three Sets, Three Roles

A robust ML workflow keeps three completely separate data pools with distinct purposes.

Set	Purpose
Training set	Fit the model's parameters. Use freely.
Validation set	Tune hyperparameters and compare candidate models. Use iteratively.
Test set	Report final generalisation performance. Use exactly once, at the end.

The Golden Rule

No information from the test set should influence any modelling decision — not feature selection, not hyperparameter tuning, not architecture choice. The moment test-set feedback shapes a decision, that set's performance estimate becomes optimistically biased.

✓ Evaluate candidate models on the validation set. Reserve the test set for the final, irrevocable report.

✗ Using the test set repeatedly to compare models turns it into a de-facto tuning set — contaminating your final estimate.

Common Code Pitfall

A frequent bug: calling model.predict() on the wrong split — for example passing test data when you intended validation data. Always double-check which variable (X_val vs X_test) is passed to predict(). Similarly, keep metric calls consistent: pass ground-truth labels first, predictions second, matching the API convention.

💡 model.score() on a LinearRegression returns R^2, not MSE. Know which built-in returns which metric.

Topic 3 — LinearRegression Parameters

fit_intercept

The intercept (bias term) shifts the regression hyperplane away from the origin. The decision to include it hinges on whether the features are centered (mean ≈ 0).

Setting	When to Use
fit_intercept=True	Default. Learns an offset. Appropriate whenever any feature has a non-negligible mean.
fit_intercept=False	Assumes the data is pre-centered. Only safe if all feature means are genuinely close to zero.

💡 Even a single feature with a notable non-zero mean is sufficient reason to leave fit_intercept=True. Disabling it prematurely introduces systematic bias.

n_jobs — Parallelism

n_jobs controls how many CPU cores scikit-learn uses. For LinearRegression, parallel speedup is not unconditional — it becomes meaningful under a specific combination of conditions.

- The problem must be large enough to offset the overhead of spawning workers.
- The feature matrix should be stored in sparse format.
- There must be multiple target columns (multi-output regression).

📌 Setting n_jobs=-1 without these conditions may not produce any speedup — or may even slow things down due to overhead.

positive

positive=True constrains all learned coefficients to be non-negative — useful in domains where negative slopes are physically impossible. However, this constraint is only implemented for dense input arrays. Passing a sparse matrix alongside positive=True will raise an error.

✗ Do not combine positive=True with sparse input. They are mutually exclusive in scikit-learn's implementation.

copy_X and Memory

copy_X controls whether the input matrix is copied before fitting (to protect the original from in-place modifications). This is a memory-management parameter and has no bearing on the intercept decision or coefficient values.

Post-Fit Attributes

After calling fit(), several attributes become available. Their presence and shape depend on the training setup.

Attribute	Availability & Shape
coef_	1-D array (n_features,) for single target. 2-D array (n_targets, n_features) for multi-output.
intercept_	Scalar for single target; 1-D array for multi-output.
n_features_in_	Count of features seen during fit.
feature_names_in_	Only present if training input was a named DataFrame.
rank_ / singular_	Only computed when X is dense. Accessing these after fitting on sparse input causes an AttributeError.

📌 The shape of coef_ is a direct readout of the training setup: (3, 4) means 3 target columns and 4 input features — classic multi-output regression.

Topic 4 — Pipelines & Data Leakage

What is Data Leakage?

Leakage occurs when the model indirectly accesses information it should not have during training — most commonly by fitting preprocessing steps (scalers, imputers, encoders) on data that includes validation or test observations. The model then appears to generalise well because it has already 'seen' those samples, even implicitly.

The Correct Preprocessing Workflow

- Split the data first — before any preprocessing is fitted.
- Fit all preprocessing (e.g., StandardScaler, SimpleImputer) exclusively on the training fold.
- Apply (transform only) those fitted preprocessors to validation and test folds.
- Wrap the preprocessor and the estimator together in a Pipeline.

A Pipeline ensures that fit() on training data and transform() on unseen data happen correctly in sequence — and crucially, that the same transformation is applied identically at inference time.

✓ Split → fit preprocessing on train → Pipeline → validate → test once.

✗ Fitting a scaler on the entire dataset before splitting leaks validation/test statistics into training. Performance estimates will be optimistically inflated.

✗ Fitting separate preprocessing per subset means each set is transformed by its own statistics, not training statistics — inconsistent and wrong.

💡 Inspecting test results and then revising preprocessing is a particularly insidious form of leakage. The test set must remain sealed until all decisions are final.

Saving and Loading Models

Persisting a model with joblib.dump / joblib.load does not change the evaluation contract. A loaded model should be called with predict() directly on the held-out test set — never refitted on it. Refitting on test data discards the trained parameters and defeats the purpose of the saved model.

Topic 5 — Problem Framing & Feature Engineering

Matching Model Type to the Decision

Before writing a single line of code, the team must ask: what output does the deployed system actually produce? A model that predicts a continuous number is different from one that produces a binary decision — even if the underlying data is the same.

Model Type	Use When...

Regression	Predicts a continuous value. Suited when the downstream consumer needs a number (e.g., estimated cost).
Classification	Predicts a category. Suited when the downstream action is discrete (e.g., approve / flag for review).

If the deployment decision is binary — for example, send an application for manual review versus pass it automatically — the model type should be classification, and the evaluation metrics should reflect that (precision, recall, F1, AUC). Forcing a regression model through a post-hoc threshold is a mismatch between model and purpose.

💡 Evaluation metrics must match the model type and the operational goal. R^2 is appropriate for regression; accuracy, recall, and F1 are appropriate for classification. Mixing them leads to misleading conclusions.

Feature Validity — Temporal Integrity

A feature is only valid if it is available at the moment the model makes its prediction in the real world. Features that are generated after the event being predicted — no matter how correlated — constitute temporal leakage and must be excluded.

- Identifier columns (IDs, record numbers) carry no predictive signal and should be dropped.
- Features derived from outcomes that happen after the prediction moment must be removed.
- Only features observable before or at the time of prediction are legitimate inputs.

📌 Ask: 'Would I have this value in my hands at the moment I need to make the prediction?' If no, it does not belong in the feature set.

Topic 6 — Model Evaluation & Error Interpretation

Confusion Matrix Fundamentals

Every binary classifier produces four types of outcomes. Understanding them is essential for choosing between models in high-stakes contexts.

Outcome	Meaning
True Positive (TP)	Correctly predicted positive — model said yes, ground truth is yes.
True Negative (TN)	Correctly predicted negative — model said no, ground truth is no.
False Positive (FP)	Model said yes, ground truth is no. Also called a false alarm.
False Negative (FN)	Model said no, ground truth is yes. Also called a miss.

When Accuracy Misleads

Accuracy measures the fraction of all predictions that were correct. It is a useful summary but can be deeply misleading when the cost of different error types is unequal, or when one class dominates.

Example: A screening test where missing a positive case (FN) is far more serious than a false alarm (FP). A model with lower overall accuracy but far fewer false negatives may be decisively preferable to a high-accuracy model that misses many positive cases.

✓ Choose the model whose error profile best matches the operational risk tolerance — not necessarily the one with the highest accuracy.

Metric	When to Prioritise
Recall (Sensitivity)	TP / (TP + FN). Measures how many actual positives were caught. Prioritise when missing a positive is costly.
Precision	TP / (TP + FP). Measures how trustworthy positive predictions are. Prioritise when false alarms are costly.
F1 Score	Harmonic mean of precision and recall. Useful when both matter.
Accuracy	(TP + TN) / All. Useful only when classes are balanced and errors are equally costly.

The Test Set Contamination Trap (Revisited)

Selecting a model by iterating on test-set feedback — even subtly, such as choosing between two models based on test score — renders the final reported number unreliable. True generalisation performance can only be estimated on data that had zero influence on any modelling decision.

✗ Checking the test set after every small adjustment and keeping the result that looks best is equivalent to overfitting to the test set.

Quick-Reference Summary

#	Topic	Core Principle to Remember
1	Stratified Splitting	Rare classes require stratify=y at every split stage independently.
2	Reproducibility	Lock your split with random_state; always split X and y together.
3	Val vs Test	Validation = tune freely. Test = open once, final report only.
4	Metric Calls	Ground truth first, predictions second. Know what score() returns.
5	fit_intercept	One non-centered feature → enable the intercept. When in doubt, leave it on.

6	n_jobs Speedup	Meaningful only: large problem + sparse X + multiple targets.
7	positive + sparse	Mutually exclusive. positive=True demands dense input.
8	Sparse Attributes	rank_ and singular_ are unavailable after fitting on sparse data.
9	coef_ Shape	1-D = single target. 2-D (n_targets, n_features) = multi-output.
10	feature_names_in_	Only populated when training input is a named DataFrame.
11	Pipeline Rule	Split first → fit preprocessor on train only → Pipeline everything.
12	joblib Load	Reload → predict → compare. Never refit on test data.
13	Temporal Leakage	Use only features available before or at prediction time. Drop IDs and post-event features.
14	Model Type Match	Binary decision → classifier. Regression for continuous output. Match metrics accordingly.
15	Error Asymmetry	Higher accuracy ≠ better model. Identify which error (FP or FN) is costlier and choose accordingly.

Final Reminders

• Understand the why, not just the what — most questions test reasoning, not memorisation.
• When a question involves code, trace through it mentally: what data goes in, what comes out?
• For real-world scenarios, always ask: Is this available at prediction time? Does this match the deployment decision?
• Trust your understanding of the principles above over pattern-matching to specific examples.

Good luck. You have prepared well. GOD BLESS.